PANADERO

Database Design and SQL Queries

ER Diagram • Schema Script • Query Demonstration

University of Cabuyao • College of Computing Studies • Version 1.0 • 12 October 2026

IMPORTANT SQL COMPLETENESS NOTE

The current database/panadero.sql defines the database and all nine tables, keys, and relationships, but contains zero INSERT statements. server.js seeds 4 categories, 15 products, and 2 demo Admin/Staff users at runtime. This still misses the assignment rule of at least five sample records in each main table (categories: 4; users: 2). The schema is present; the sample-data portion of the submitted SQL script is not complete.

Schema overview

Database: panadero_bakery. The current schema has nine InnoDB tables and exceeds the six-table minimum. Main entities are users, categories, and bread_products. Orders, reservations, and inventory_batches represent operational/transaction records. order_items resolves the many-to-many relationship between orders and products.

Table	Primary key	Foreign key(s)	Purpose
users	user_id	—	Customer, staff, and admin accounts.
categories	category_id	—	Product category definitions.
bread_products	product_id	category_id → categories	Bakery products, prices, stock.
orders	order_id	user_id → users	Customer order header, total, status, date.
order_items	order_item_id	order_id → orders; product_id → bread_products	Order-product junction with quantity and unit price.
reservations	reservation_id	user_id → users; product_id → bread_products	Scheduled product reservation details/status.
inventory_batches	batch_id	product_id → bread_products	Baked quantity, dates, and batch notes.
system_logs	log_id	user_id → users (nullable)	System action type/description/date.
customer_notifications	notification_id	user_id → users	Unique per-customer completion notification state.

Relationship highlights

- One category has many bread_products; each product references one category.
- One user can have many orders and reservations; each request belongs to a user.
- orders and bread_products are many-to-many through order_items (order_id + product_id foreign keys).
- Products can have many inventory_batches and reservations; user notifications reference their owner.

Every table declares a primary key. The DDL includes more than four foreign-key relationships. Foreign-key delete/update actions are defined in database/panadero.sql.

Entity-Relationship Diagram

The diagram below was rendered from the current nine-table SQL schema; PK and FK markers identify keys and the connectors show declared relationships.

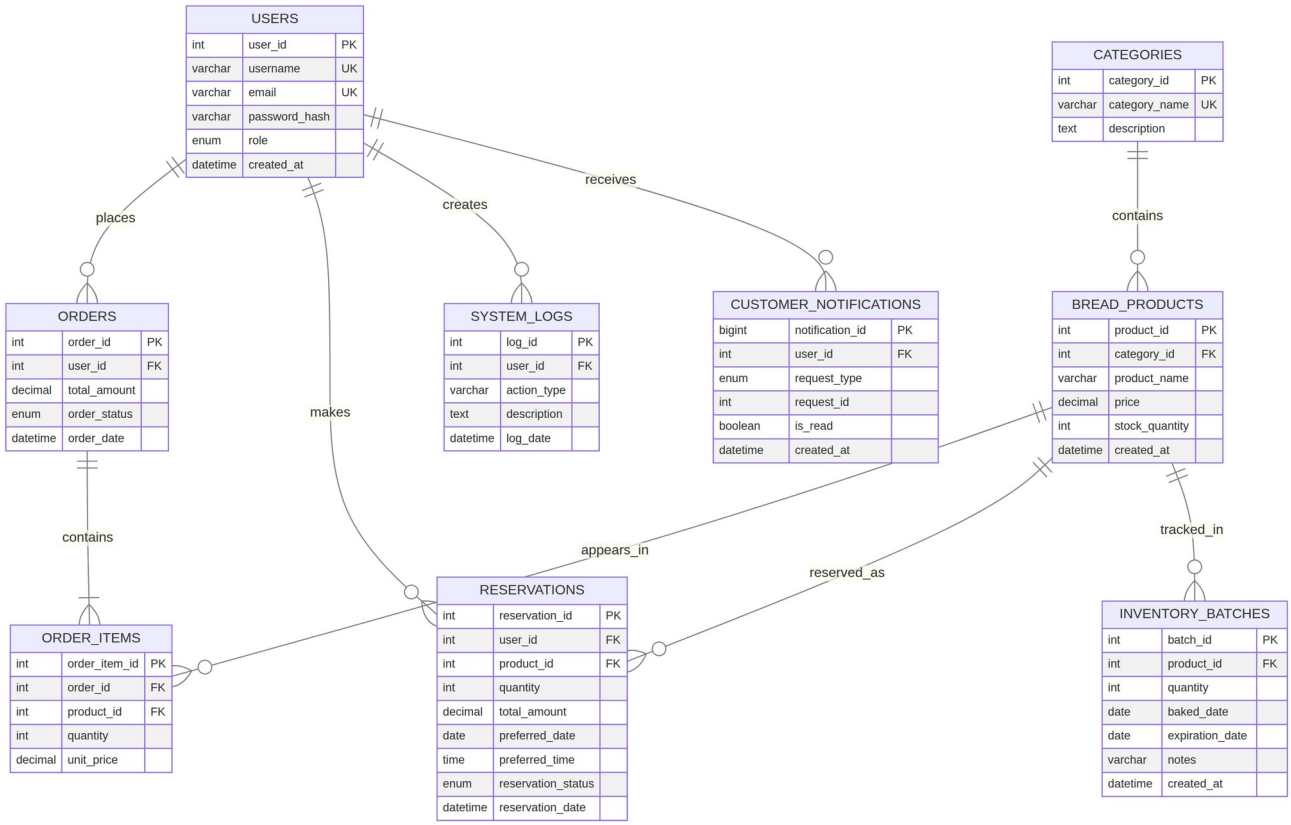


Figure 1. PANADERO nine-table ERD (generated from database/panadero.sql).

SQL Query Demonstration

The assignment asks for SELECT, INSERT, UPDATE, and DELETE, plus an advanced query that combines WHERE, ORDER BY, and GROUP BY or JOIN. The examples use the current schema and include their purpose and expected result. They were not run against a live MySQL instance during this documentation pass; test them on a disposable database copy before presentation.

1. SELECT — list available products

```
SELECT p.product_id, p.product_name, c.category_name, p.price, p.stock_quantity
FROM bread_products AS p
JOIN categories AS c ON c.category_id = p.category_id
WHERE p.stock_quantity > 0
ORDER BY c.category_name, p.product_name;
```

Purpose / expected output: returns in-stock products with their category, price, and quantity, ordered by category and product name.

2. INSERT — add a test category and product

```
START TRANSACTION;
INSERT INTO categories (category_name, description)
VALUES ('Seasonal Test', 'Temporary query demonstration');
SET @test_category_id = LAST_INSERT_ID();
INSERT INTO bread_products (category_id, product_name, price, stock_quantity)
VALUES (@test_category_id, 'Seasonal Roll Test', 75.00, 10);
COMMIT;
```

Purpose / expected output: creates one temporary category and one product and stores the new category ID in the current SQL session. Run only on a test database; the later DELETE example removes these test rows.

3. UPDATE — adjust product stock

```
UPDATE bread_products
SET stock_quantity = stock_quantity + 10
WHERE product_id = 1;
```

Purpose / expected output: increases product 1 stock by ten; expected affected rows: one if that product exists.

4. DELETE — remove the temporary test product/category

```
DELETE FROM bread_products
WHERE product_name = 'Seasonal Roll Test'
  AND category_id = @test_category_id;
DELETE FROM categories
WHERE category_id = @test_category_id;
```

Purpose / expected output: removes the test product first, then its now-unreferenced test category; expected affected rows: one per statement if the INSERT example ran in the same session.

5. Advanced — 30-day product sales summary

```
SELECT c.category_name, p.product_name,
       SUM(oi.quantity) AS units_sold,
       SUM(oi.quantity * oi.unit_price) AS gross_sales
FROM orders AS o
JOIN order_items AS oi ON oi.order_id = o.order_id
JOIN bread_products AS p ON p.product_id = oi.product_id
JOIN categories AS c ON c.category_id = p.category_id
WHERE o.order_date >= CURRENT_DATE - INTERVAL 30 DAY
GROUP BY c.category_name, p.product_id, p.product_name
ORDER BY gross_sales DESC, units_sold DESC;
```

Purpose / expected output: joins orders, order_items, products, and categories; filters to the last 30 days; groups sales per product and orders by gross sales. It combines WHERE, JOIN, GROUP BY, and ORDER BY as required.

Appendix A — Canonical SQL Schema Script

The following is the current database/panadero.sql schema file. It includes CREATE DATABASE and CREATE TABLE statements for all nine tables, primary keys, foreign keys, and constraints. Its final comment notes that runtime categories/products/demo accounts are seeded by server.js; the file itself does not yet contain the required sample INSERT dataset.

```
CREATE DATABASE IF NOT EXISTS panadero_bakery;
USE panadero_bakery;

CREATE TABLE IF NOT EXISTS users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    email VARCHAR(100) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role ENUM('Admin', 'Staff', 'Customer') NOT NULL DEFAULT 'Customer',
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE,
    description TEXT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS bread_products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES categories(category_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    order_status ENUM('Pending', 'Processing', 'Ready', 'Completed', 'Cancelled') NOT NULL DEFAULT 'Pending',
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS order_items (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
        ON DELETE CASCADE ON UPDATE CASCADE,
    FOREIGN KEY (product_id) REFERENCES bread_products(product_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS reservations (
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    total_amount DECIMAL(10,2) NOT NULL,
    preferred_date DATE NOT NULL,
    preferred_time TIME NOT NULL,
    reservation_status ENUM('Pending', 'Processing', 'Ready', 'Completed', 'Cancelled') NOT NULL DEFAULT 'Pending',
    reservation_date DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (product_id) REFERENCES bread_products(product_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS inventory_batches (
    batch_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    baked_date DATE NOT NULL,
    expiration_date DATE NOT NULL,
    notes VARCHAR(500) NULL,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES bread_products(product_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS system_logs (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    action_type VARCHAR(50) NOT NULL,
    description TEXT NULL,
    log_date DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;


-- Stored per-customer completion notifications (safe on an existing PANADERO database).
CREATE TABLE IF NOT EXISTS customer_notifications (
    notification_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    request_type ENUM('order', 'reservation') NOT NULL,
    request_id INT NOT NULL,
    is_read TINYINT(1) NOT NULL DEFAULT 0,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
    UNIQUE KEY unique_customer_completion (user_id, request_type, request_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- Categories, products, and demo staff/admin accounts are seeded by server.js on startup.
```

Sample-data checklist to complete separately: provide at least five INSERT sample records for each main table (users, categories, and bread_products) in the submitted SQL script; use safe classroom-only accounts/password hashes and verify foreign-key order. The runtime starter data is not a substitute for explicit INSERT statements in the SQL deliverable.